

NCCL

- NVIDIA Collective Communications Library
- de-facto standard for multi-GPU communication in deep learning
- GPU-to-GPU data transfers without CPU involvement
- supports GPUs on a single and multiple nodes
- implements collective communication primitives for GPUs
- key design goals
 - high bandwidth utilization of available interconnects
 - topology-aware communication — routes data through the fastest path
 - asynchronous execution via CUDA streams
- API designed similarly to MPI collectives
 - collective and point-to-point communication
 - no so many functions as MPI
 - NCCL data types: `ncclFloat16`, `ncclFloat32`, `ncclFloat64`, `ncclInt32`, `ncclInt64`, `ncclBfloat16`

Topology Detection

- NCCL automatically detects the GPU interconnect topology at communicator initialization
- topology determines which communication algorithm and transport to use
- detected interconnects (in order of preference):
 - NVLink — NVIDIA's proprietary high-bandwidth GPU interconnect
 - NVLink 4.0 (Hopper): up to 900 GB/s - 18 connections, capacity of each 50 GB/s
 - PCIe
 - limited by PCIe bandwidth (up to 256 GB/s)
 - NVSwitch
 - all-to-all NVLink switch in DGX systems
 - connects all GPUs in a node at full NVLink bandwidth
 - InfiniBand / Ethernet
- topology is represented internally as a graph
 - nodes: GPUs, CPUs, NICs, NVSwitches
 - edges: interconnects with associated bandwidth and latency
 - NCCL selects paths that maximize bandwidth and minimize hops

Data Transport

- NCCL selects the best transport per GPU pair
- three transport types, chosen automatically:
 - peer-to-peer — used when GPUs are connected via NVLink or can do PCIe peer access
 - data is read/written directly between GPU memory spaces by the GPU's direct memory access engine
 - CPU is not involved in moving the data at all
 - shared memory — used when GPUs cannot use peer-to-peer
 - data is staged through a shared CPU memory buffer
 - CPU memory acts as an intermediate hop; still faster than full CPU processing
 - network — used for multi-node communication
 - remote direct memory access protocols
 - if not available, data is copied to the host memory first then sent
 - the network interface card reads/writes GPU memory directly without copying through CPU or host RAM
- pipelining
 - the data buffer is divided into chunks
 - chunks are pipelined to hide latency and keeps interconnect bandwidth saturated

Communication Algorithms

- NCCL selects the algorithm automatically based on topology and message size
- NCCL switches between algorithms automatically depending on message size and GPU count

Programing Interface

Communicators

- defines a group of GPUs that participate in collective operations (similar to MPI)
- each GPU in the group holds its own communicator handle
- structure `ncclComm_t`
- communicator GPU rank in range `[0, number of GPUS - 1]`

CUDA Streams and Asynchronous Execution

- all NCCL operations are enqueued into a CUDA stream
 - they execute asynchronously with respect to the CPU
 - the CPU does not block while the collective runs
- synchronization is done explicitly
 - `cudaStreamSynchronize(stream)` — wait for a specific stream
- allows overlapping computation and communication (kernels on one stream, NCCL on another stream)

Group Operations

- when running multiple GPUs from a single CPU thread, NCCL operations must be grouped
- `ncclGroupStart()` and `ncclGroupEnd()` mark a group of operations
 - calls inside the group are batched and launched together
 - required for multi-GPU single-process usage
 - optional but recommended for multi-process usage to improve performance
- `ncclAllReduce` is an example of group operation

Example: Allreduce via Host and via NCCL Comparison

- single-node, 2-GPU AllReduce comparing NCCL with a naive host-based implementation
- [allreduceNCCL.cu](#) and [allreduceNCCL.sh](#)